Desenvolvimento de Microserviços

Prof. Guilherme Alves

FACULDADE Donaduzzi

O **NestJS** é um framework para desenvolvimento de aplicações backend em **Node.js**, focado em escalabilidade, organização e arquitetura modular. Ele é construído sobre **Node.js** e utiliza **TypeScript** como linguagem principal. Internamente, o **NestJS** utiliza frameworks HTTP como: **Express** e **Fastify.**

express

node
JS
TM

Requisitos

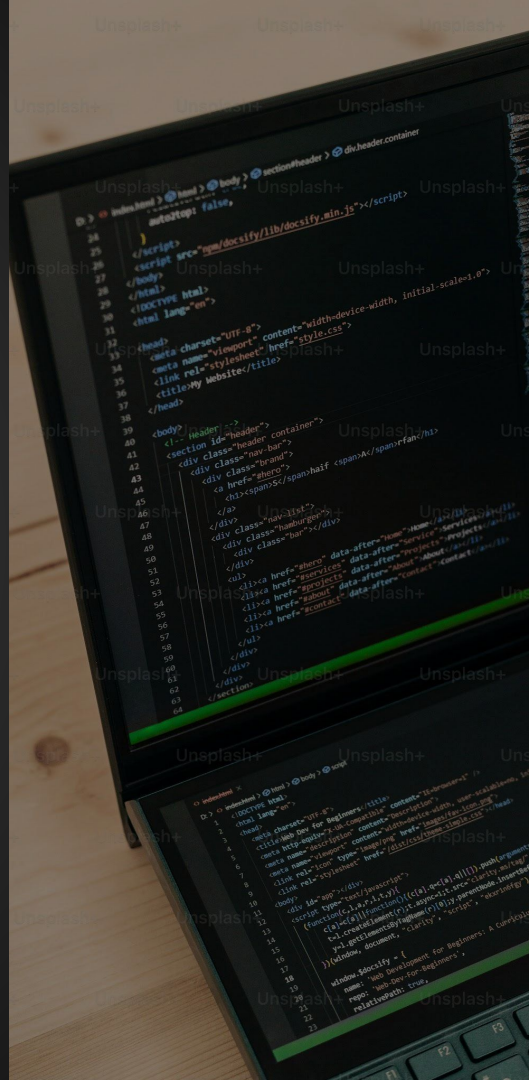
Node.js > 20

$ node -v

$ npm -v

$ npm install -g @nestjs/cli

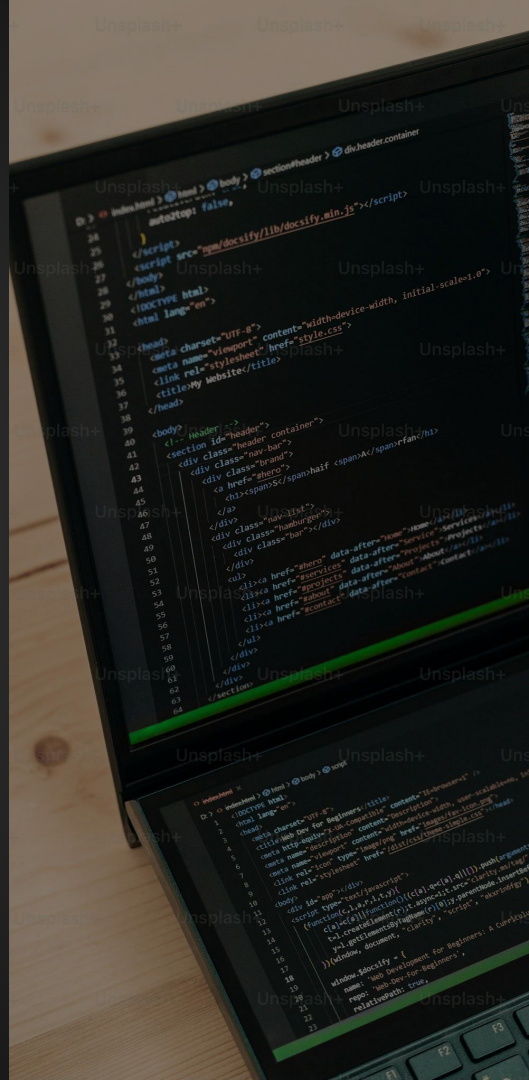
$ nest -v



Criação do primeiro projeto

$ nest new nome-do-projeto

```
src
├── app.controller.ts   — define rotas
├── app.module.ts       — módulo principal
├── app.service.ts      — lógica de negócio
└── main.ts             — ponto de entrada da aplicação
```

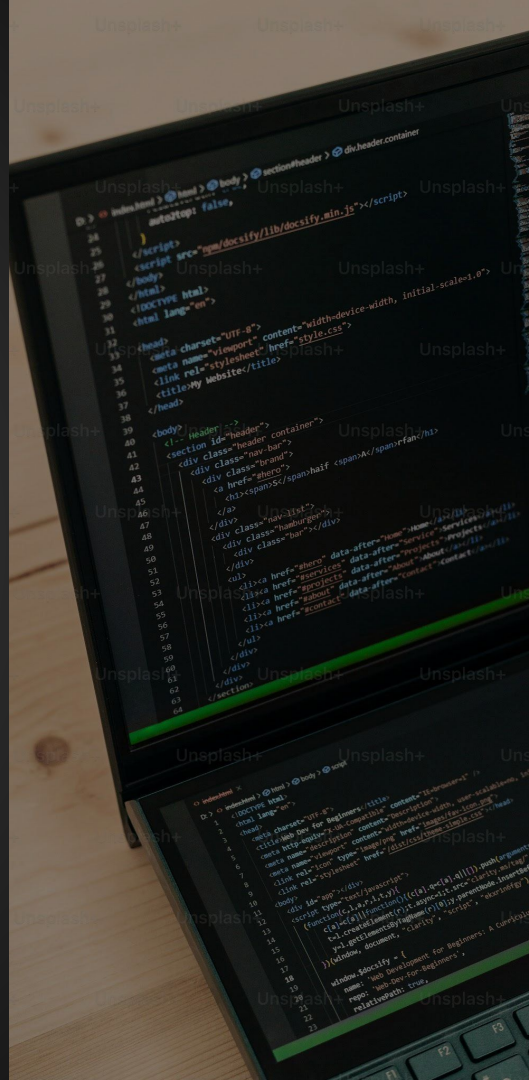


Executando o projeto

$ cd nome-do-projeto

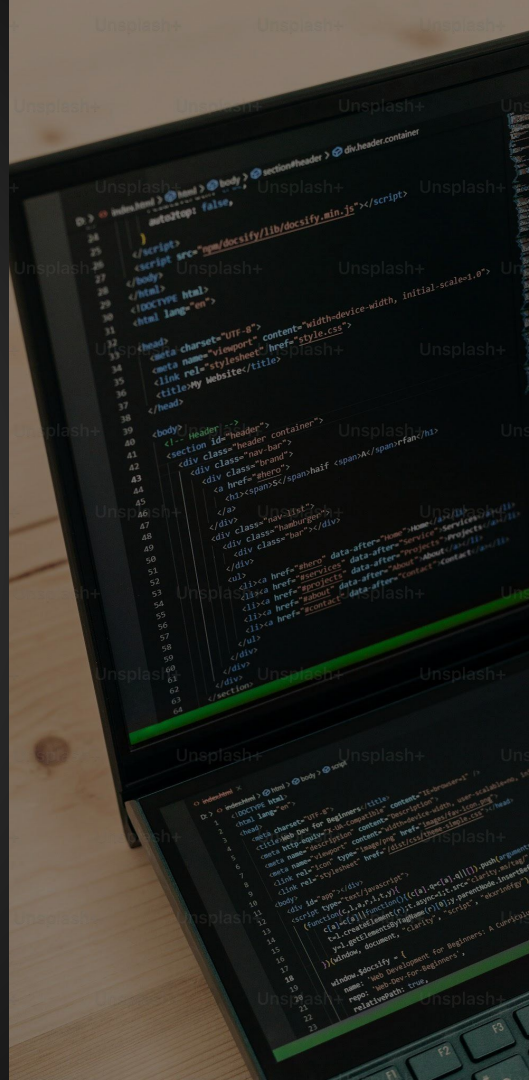
$ npm run start:dev

http://localhost:3000



Arqultetura do NestJS

- Modules
- Controllers
- Providers
- Middleware
- Guards
- Pipes
- Interceptors
- Exception Filters
- Custom Decorator

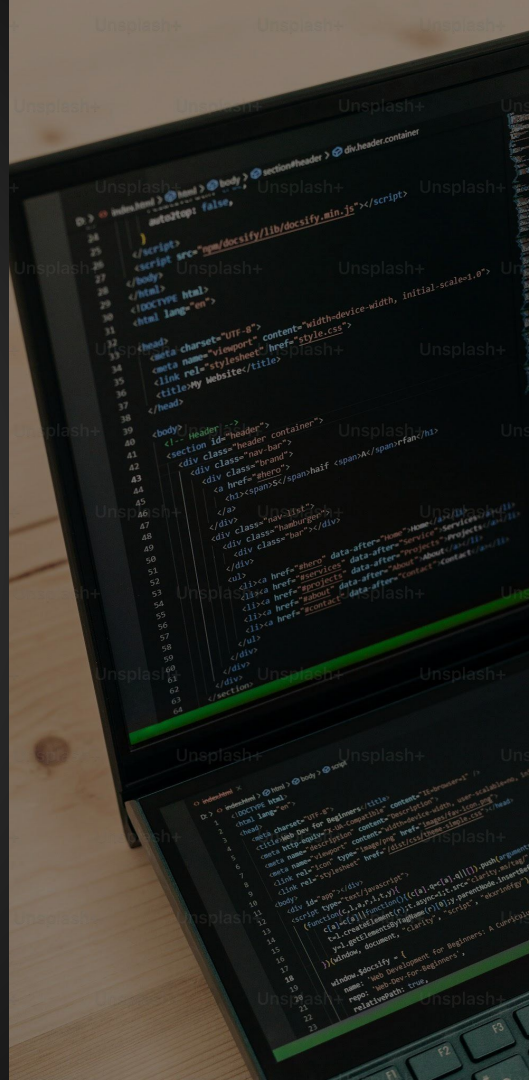


Modules

São responsáveis por organizar a aplicação em partes funcionais. Um módulo é uma classe anotada com o decorador **@Module**. Esse decorador fornece metadados que o **Nest** utiliza para organizar e gerenciar a estrutura da aplicação de forma eficiente.

Serve para delimitar as fronteiras de cada **bounded context**.

$ nest g module nome-do-modulo

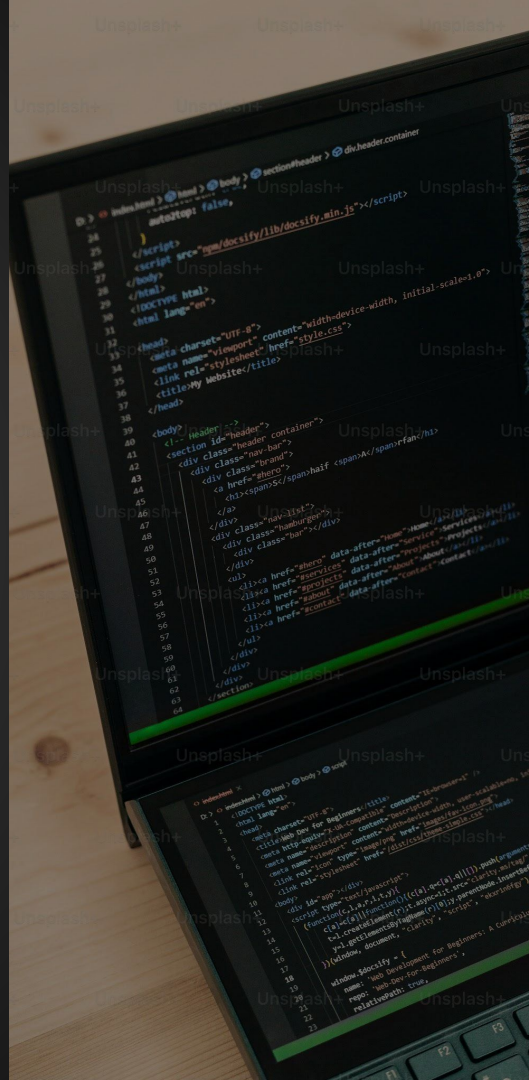


Controllers

São responsáveis por receber requisições HTTP e retornar respostas. A função de um controlador é lidar com solicitações específicas da aplicação. O mecanismo de roteamento determina qual controlador irá processar cada solicitação. Frequentemente, um controlador possui múltiplas rotas, e cada rota pode executar uma ação diferente. Representados pela anotação **@Controller**.

Serve para expor recursos de cada **bounded context**.

$ nest g controller nome-do-controller



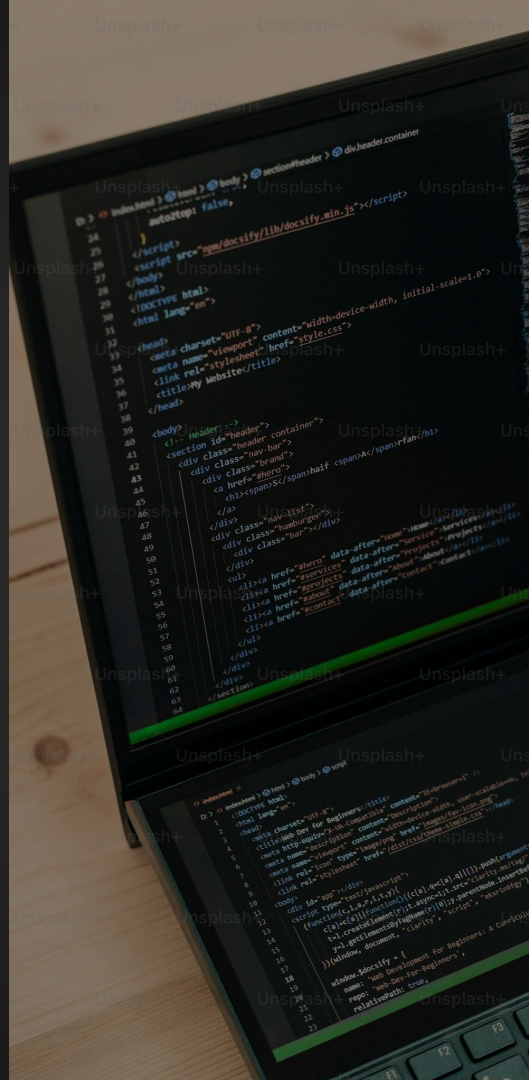
Providers

Os provedores são um conceito central no **Nest**. Muitas das classes básicas do **Nest**, como **serviços**, **repositórios**, **fábricas** e **auxiliares**, podem ser tratadas como provedores.

São classes que podem ser injetadas como dependência em outras classes. Representados pela anotação **@Injectable**.

Serve para injetar recursos de cada **bounded context**.

$ nest g service nome-da-service

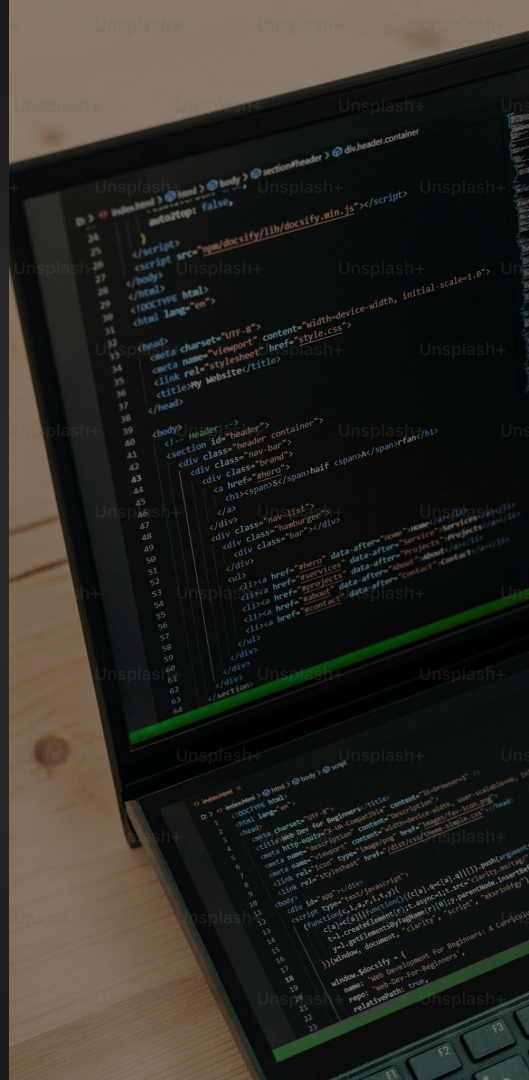


Middleware

Executa antes da requisição chegar no controller. Ele funciona de forma semelhante ao middleware do **Express**.

Implementa a interface **NestMiddleware**.

Serve para injetar recursos a rotas dos controllers.

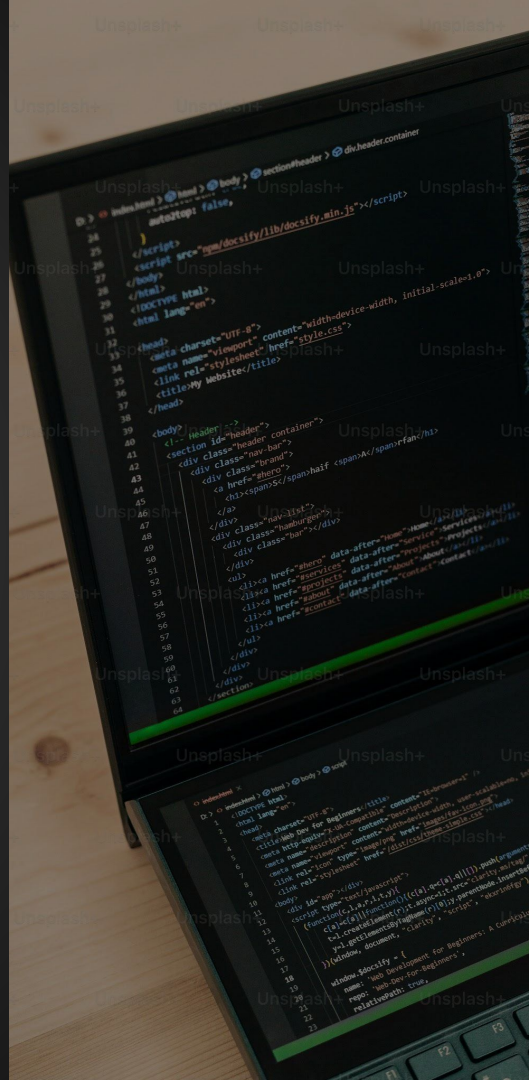


Guards

São responsáveis por controle de acesso a recursos.

Implementa a interface **CanActivate**.

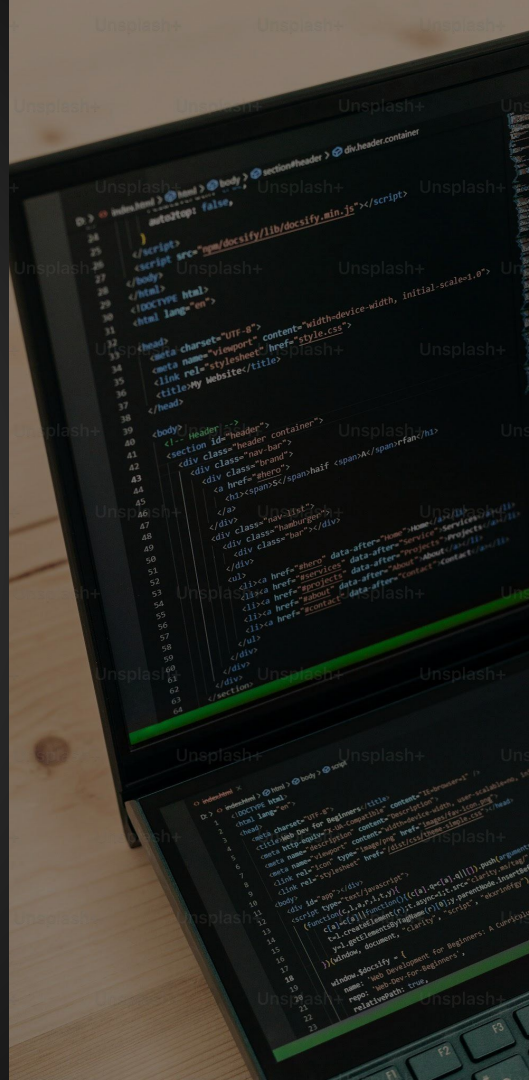
Serve para autorização de recursos a rotas dos controllers.



Pipes

São usados para validação e transformação de dados.
Eles são muito utilizados com parâmetros de requisição.

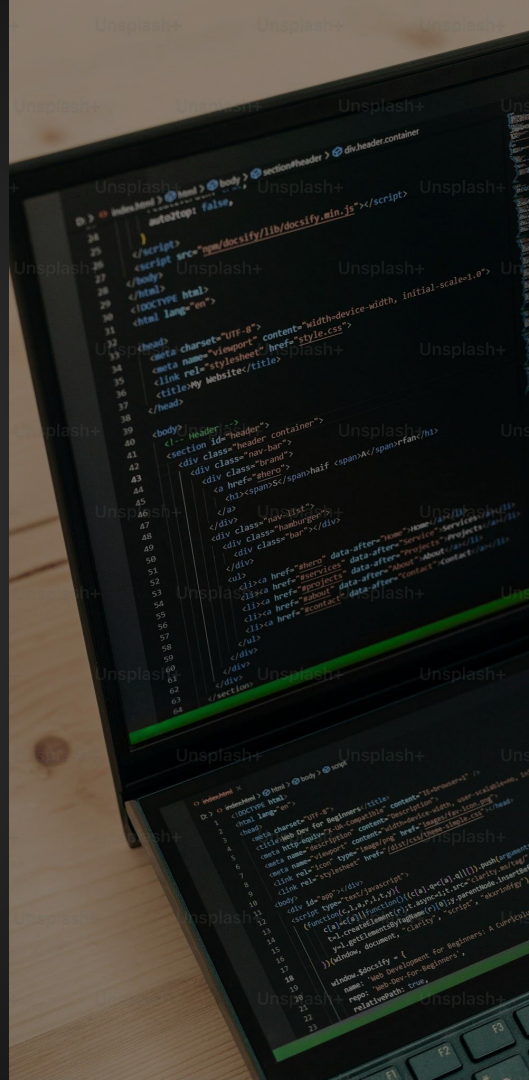
Implementa a interface **PipeTransform**.



Interceptors

Permitem interceptar requisições e respostas. Eles podem executar código antes e depois da execução do controller.

Implementa a interface **NestInterceptor**.

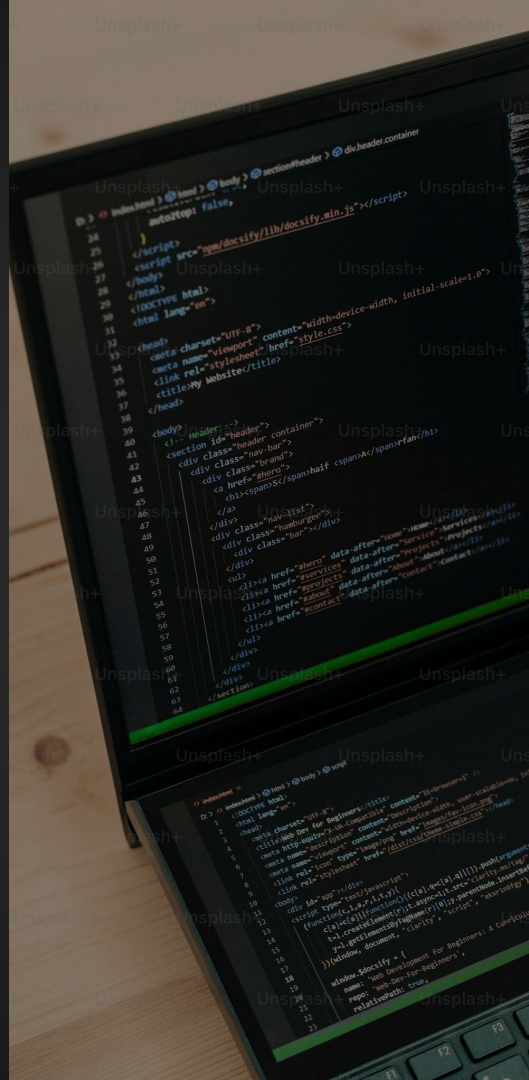


Exception Filters

São responsáveis por capturar e tratar erros da aplicação.

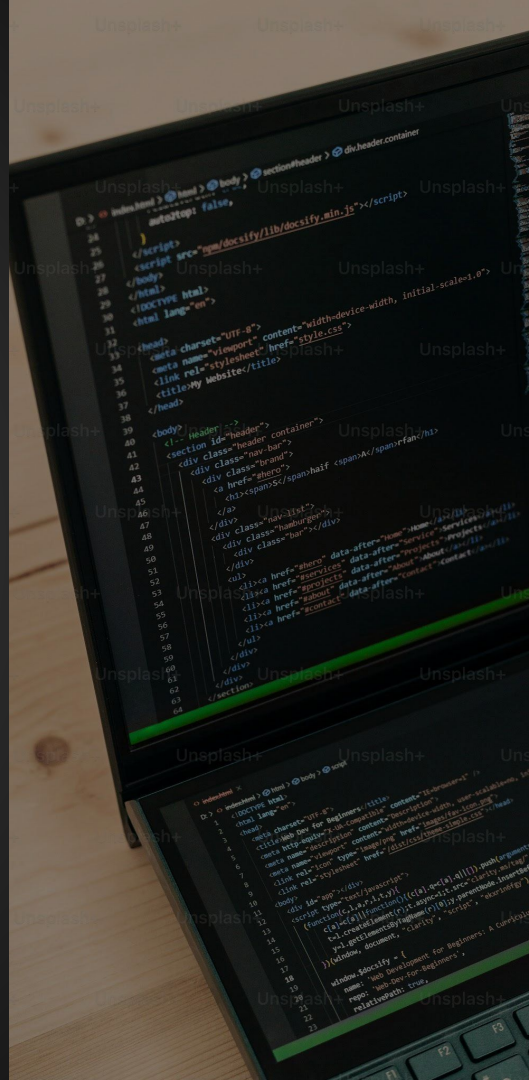
Eles permitem padronizar respostas de erro.

Representados pela anotação **@Catch**.

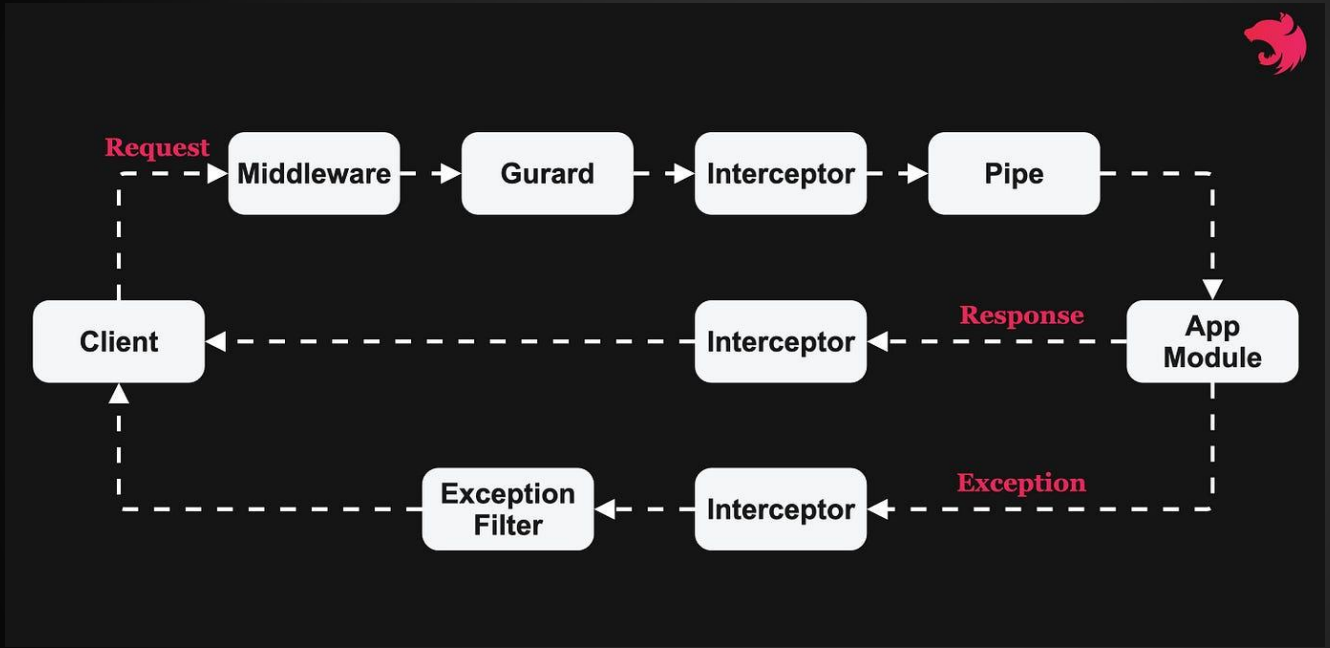


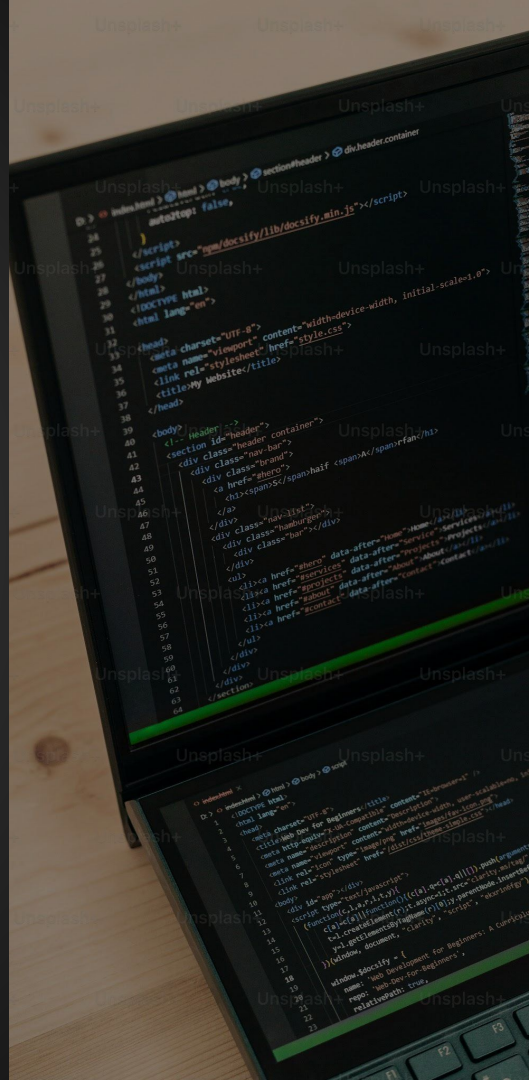
Custom Decorators

Permitem criar abstrações reutilizáveis.



Fluxo de requisição no NestJS





Uma instituição de ensino necessita de um sistema para auxiliar no controle das atividades acadêmicas relacionadas às aulas e à frequência dos alunos. Atualmente, grande parte desses processos é realizada de forma manual ou utilizando ferramentas isoladas, o que dificulta a organização das informações, o acompanhamento da frequência e a geração de relatórios confiáveis.

O sistema deve permitir o cadastro e gerenciamento das entidades acadêmicas básicas, como **alunos**, **professores** e **disciplinas**.

Cada aluno deve possuir um registro único contendo informações necessárias para sua identificação dentro da instituição. Da mesma forma, professores precisam ser cadastrados para que possam ser vinculados às disciplinas e às turmas que irão lecionar. As disciplinas representam os componentes curriculares oferecidos pela instituição e devem possuir informações como nome, código e carga horária.

Além do cadastro dessas entidades, o sistema deve possibilitar a oferta de disciplinas em forma de turmas. Uma disciplina pode ser ofertada em diferentes períodos ou semestres, podendo gerar várias turmas distintas. Cada turma deve estar associada a uma disciplina específica e possuir um professor responsável pela condução das aulas.

Após a criação das turmas, os alunos devem poder ser matriculados nessas turmas, estabelecendo o vínculo entre aluno e disciplina ofertada naquele período. Uma turma pode possuir vários alunos matriculados, e cada aluno pode estar matriculado em várias turmas ao mesmo tempo, dependendo de sua grade curricular.

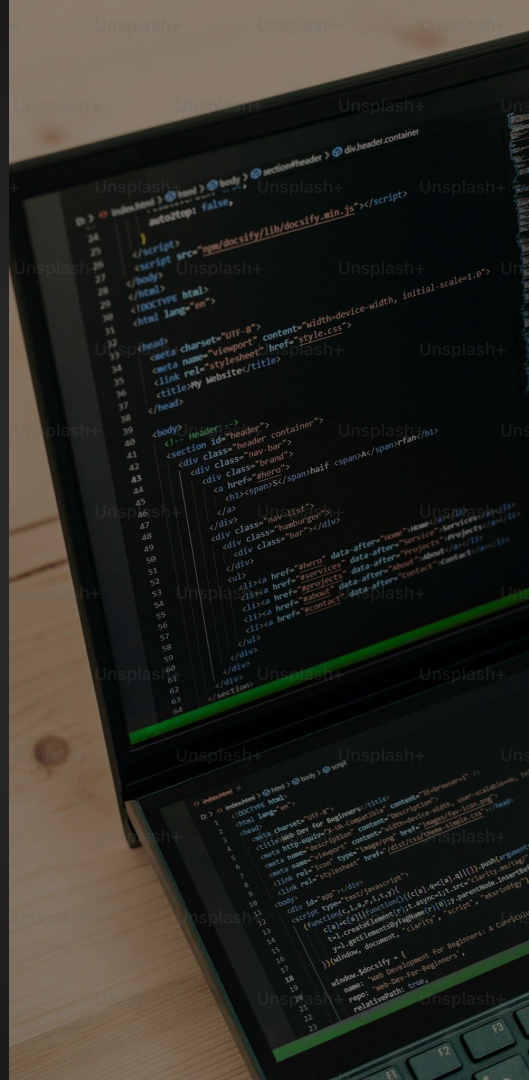
Uma vez que os alunos estejam matriculados em uma turma, o professor responsável deverá realizar o registro da chamada das aulas, informando quais alunos estiveram presentes ou ausentes em determinada data. Cada aula realizada deve possuir um registro de frequência contendo os alunos da turma e seus respectivos status de presença.

O sistema deve garantir que somente alunos matriculados em uma turma possam aparecer no registro de chamada dessa turma, evitando inconsistências nos dados. Além disso, deve permitir a consulta posterior dessas informações, possibilitando a visualização da frequência de cada aluno ao longo do período letivo.

Diante dessas necessidades, o sistema deve organizar e gerenciar informações relacionadas a cadastro acadêmico, oferta de disciplinas, matrículas e registro de frequência, garantindo consistência dos dados e facilitando o controle das atividades acadêmicas relacionadas às aulas e à presença dos alunos.

Glossário

- **Aluno** → *Student*
- **Professor** → *Teacher*
- **Disciplina** → *Subject*
- **Turma** → *ClassOffering*
- **Matrícula** → *Enrollment*
- **Aula** → *Lesson*
- **Chamada** → *AttendanceSession*
- **Presença** → *Present*
- **Falta** → *Absent*



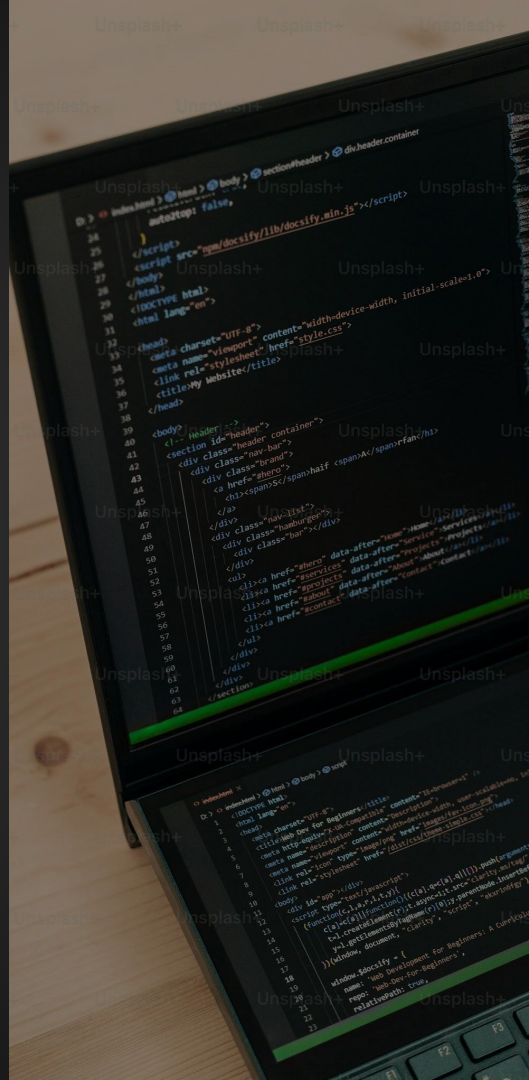
Bounded Context

```
Acadêmico
├── Aluno
├── Professor
└── Disciplina

Oferta Acadêmica
└── Turma / Oferta de Disciplina

Matrícula
└── Matrícula do aluno na turma

Frequência
└── Chamada e registro de presença
```



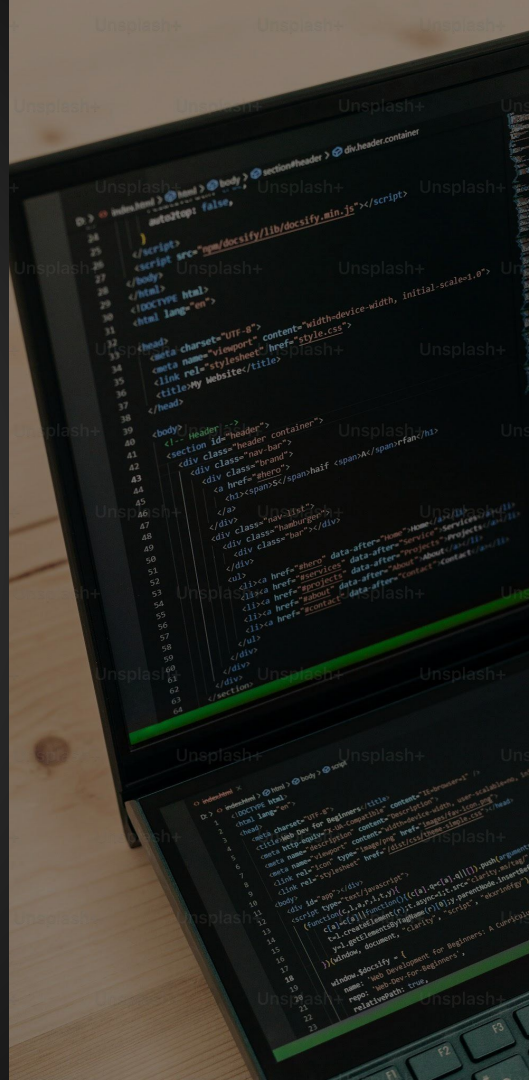
Acadêmico (*academic*)

Conceitos

- Aluno
- Professor
- Disciplina

Responsabilidades

- Cadastrar aluno
- Cadastrar professor
- Cadastrar disciplina



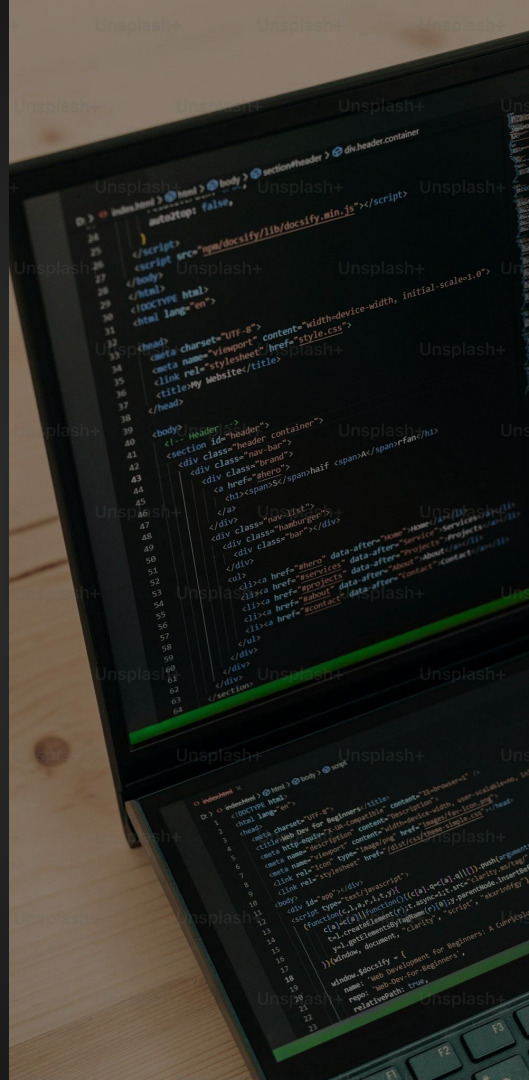
Oferta Acadêmica (*offering*)

Conceitos

- Turma
- Oferta de disciplina
- Semestre/período letivo
- Professor responsável

Responsabilidades

- Criar turma
- Vincular disciplina à turma
- Vincular professor à turma
- Definir período de oferta



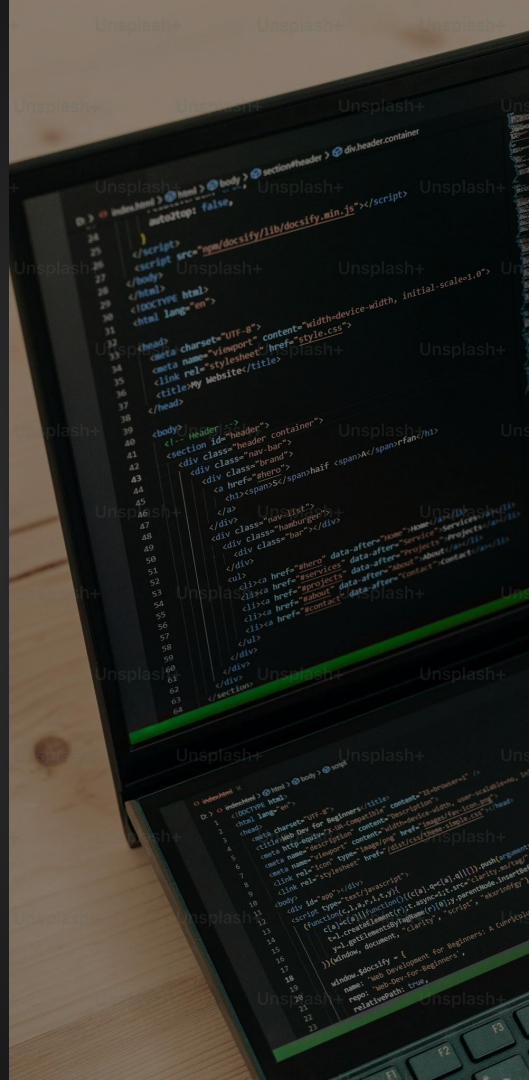
Matrícula (*enrollment*)

Conceitos

- Matrícula
- Status da matrícula
- Aluno matriculado
- Cancelamento de matrícula

Responsabilidades

- Matricular aluno em turma
- Cancelar matrícula
- Consultar alunos matriculados por turma



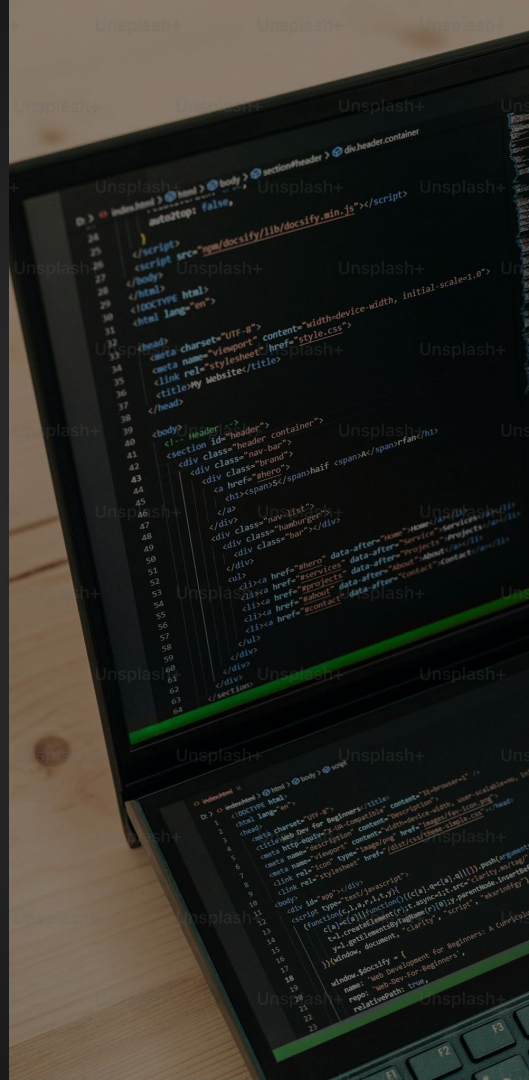
Frequência (*attendance*)

Conceitos

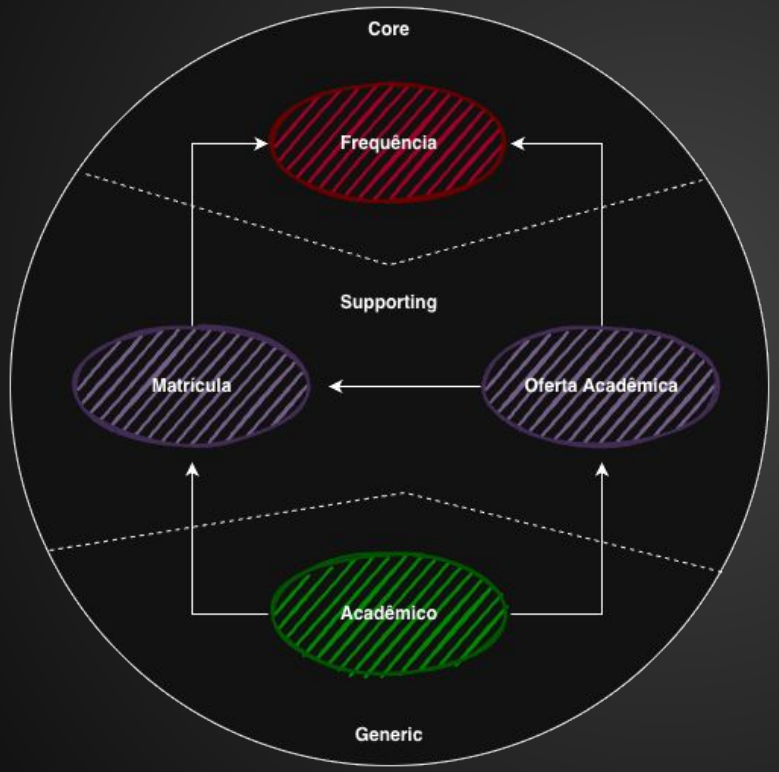
- Aula
- Registro de frequência

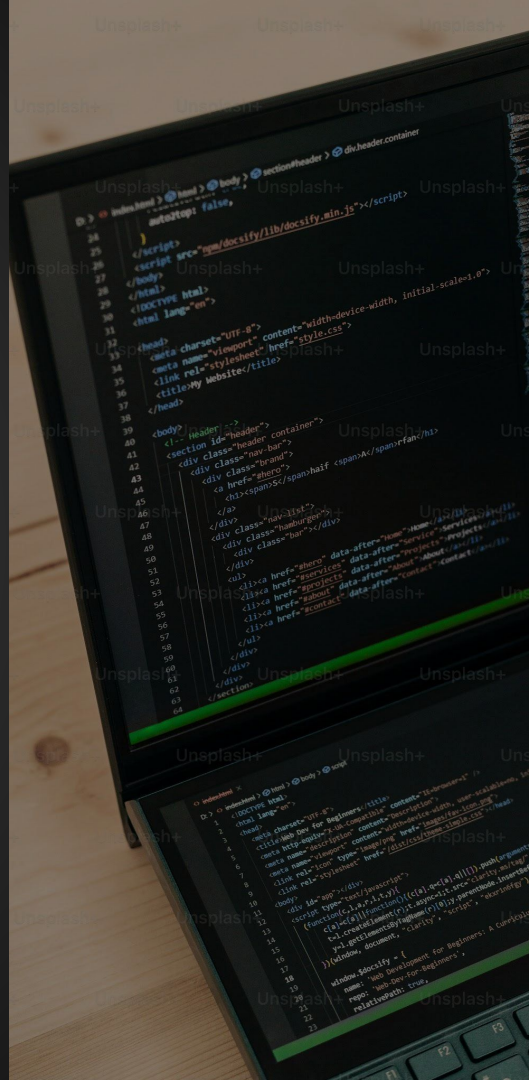
Responsabilidades

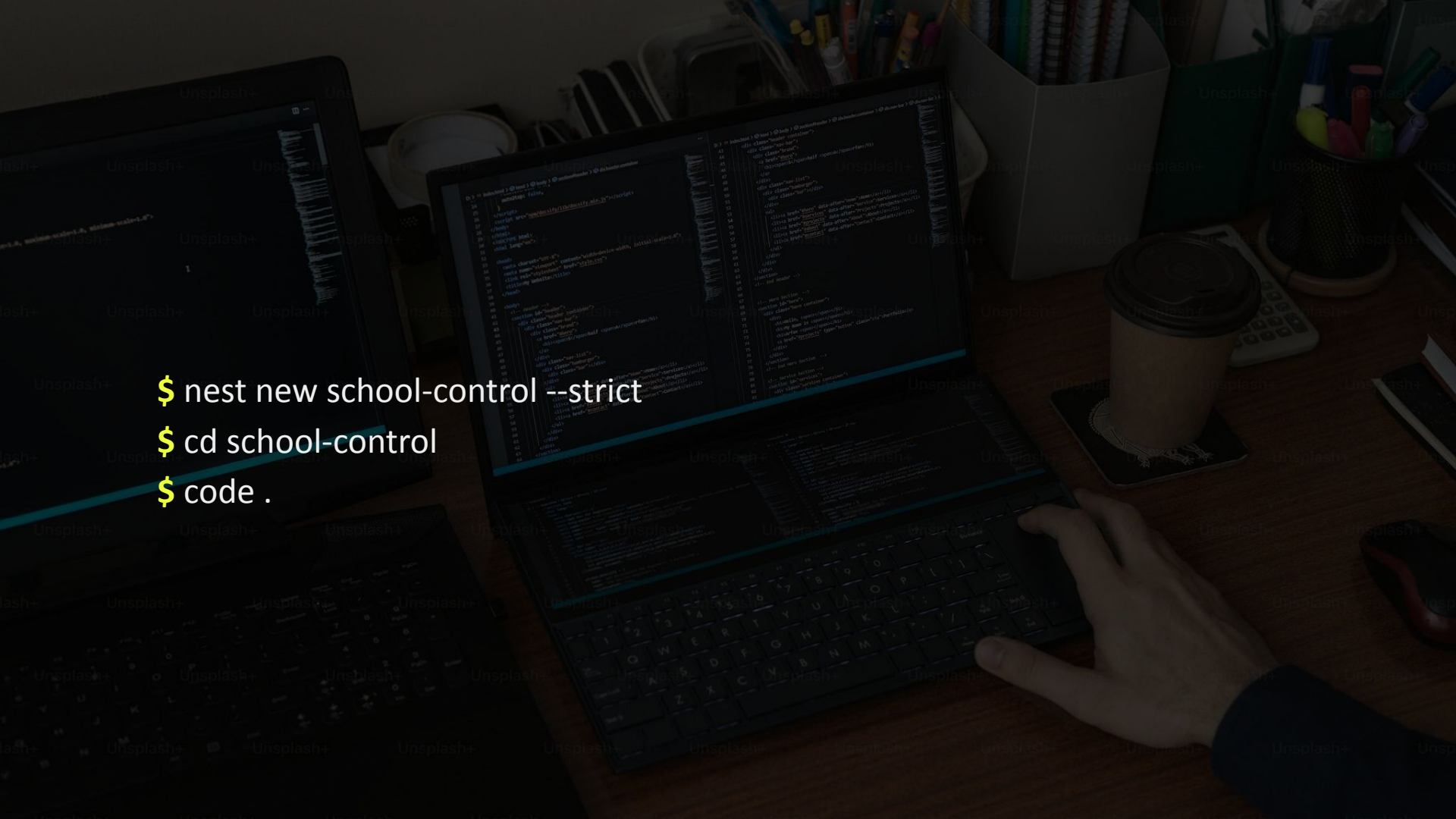
- Registrar presença ou ausência
- Consultar frequência do aluno
- Consultar frequência da turma



Context Mapping







```
$ nest new school-control --strict
$ cd school-control
$ code .
```